KNN

El juego mortal donde tus vecinos deciden tu clase... y luego descubres que hubieras preferido no tener vecinos

Daniel Barajas Higuera

MLEARN



Contenido

1. Aprendizaje Supervisado
2. Regresión
3. Clasificación
4. Prueba, ventajas y desventajas

De etiquetas y valores en el mismo tablero

En el aprendizaje supervisado hay dos caminos:

- **Regresión**: predecir un valor numérico a partir de variables de entrada.
- **Clasificación**: asignar una categoría a partir de ejemplos conocidos.

En un caso nos interesa a qué grupo pertenece una observación; en el otro, cuánto vale su resultado. La diferencia parece sencilla, pero en la práctica puede convertirse en un juego mortal donde tu clase depende de qué grupo de vecinos decida traicionarte primero.



La regla simple que decide destinos

Un participante nuevo llega con cierta edad y una deuda nada elegante. La decisión no sale de una fórmula cerrada, sino de mirar **quiénes están más cerca** en ese espacio de características.

Idea central: K-nearest neighbors busca los K vecinos más próximos y:

- en **regresión**, promedia los valores de ese vecindario.
- en **clasificación**, asigna la clase mayoritaria del vecindario;



Distancias en KNN

La noción de **cercanía** se formaliza con la **distancia de Minkowski**, que generaliza varias métricas:

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{\frac{1}{p}}$$

- $p = 1$: distancia Manhattan.
- $p = 2$: distancia Euclidiana.
- $p \to \infty$: distancia Chebyshev.

La elección de p define la forma del vecindario y puede alterar la decisión final en KNN.

Distancia Manhattan

Cuando $p = 1$, la fórmula se reduce a:

$$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{n} |x_i - y_i|$$

Se conoce como **distancia Manhattan** porque mide la longitud de un trayecto siguiendo únicamente líneas rectas en ángulos rectos, como moverse por calles en cuadrícula.



Distancia Euclidiana

Cuando $p = 2$, la fórmula es:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Es la **distancia Euclidiana**, la medida habitual de "línea recta" entre dos puntos.



Elegir K sin lotería

El tamaño del vecindario controla el sesgo-varianza:

- K pequeño: sensible al ruido, sobreajuste.
- K grande: fronteras demasiado suaves, subajuste.

Selección mediante validación cruzada y, si aplica, ponderación por distancia.

Mini-dataset Squid Game

Perfiles ficticios de jugadores: edad, deuda y tiempo de supervivencia.

```
import pandas as pd

data = {
    "Jugador": ["Joven tarjeta", "Joven apuestas",
                "Adulto hipoteca", "Adulto préstamos",
                "Mayor negocio fallido", "Mayor bancarrota"],
    "Edad": [25, 28, 45, 42, 60, 65],
    "Deuda_millones": [10, 50, 200, 120, 500, 700],
    "Supervivencia_min": [15, 12, 7, 9, 3, 2]
}
df = pd.DataFrame(data)
df
```

Contenido

1. Aprendizaje Supervisado
2. Regresión
3. Clasificación
4. Prueba, ventajas y desventajas

Regresión con KNN

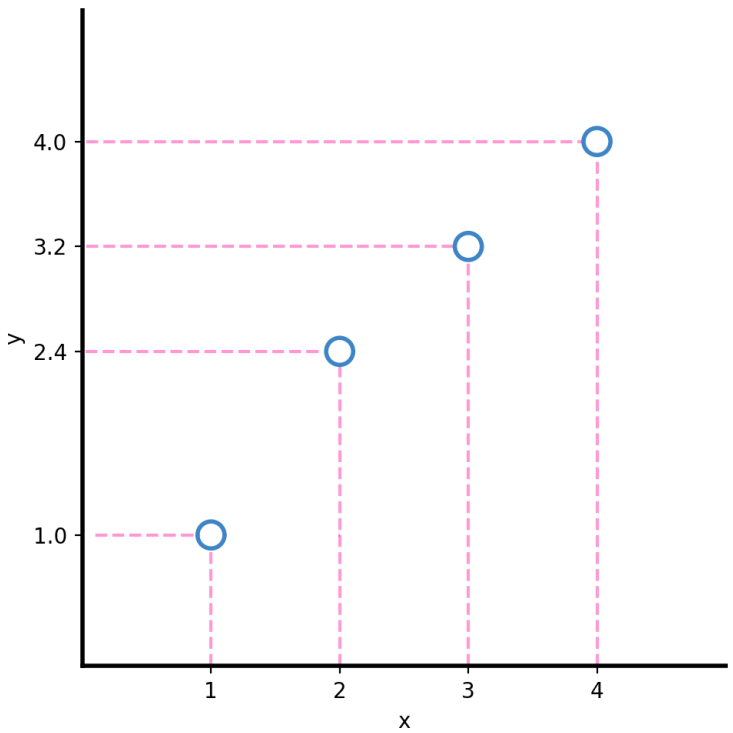


¿Cómo funciona regresión con KNN?

La idea subyacente en el algoritmo de regresión KNN es muy simple, la media de k distancias.

En este contexto, se examina un problema sencillo de regresión: cuatro puntos de datos, cada uno con una única característica representada como x y una salida representada como y.

El punto de datos cuyo valor de y se busca predecir está marcado con un ■ en el eje x.



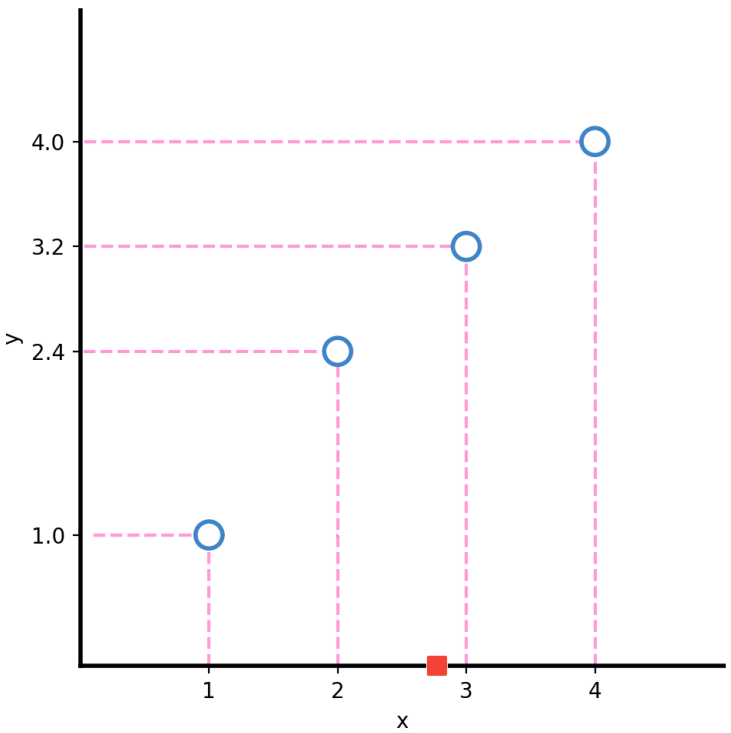
Regresión con KNN

Inicialmente, es necesario elegir un valor de K.

Se consideran tres escenarios: $K = 1, 2, 3$.

La elección de $K = 1$ busca el punto cuyo componente x se acerca más al de el ■. En este caso, es el tercer punto, de izquierda a derecha.

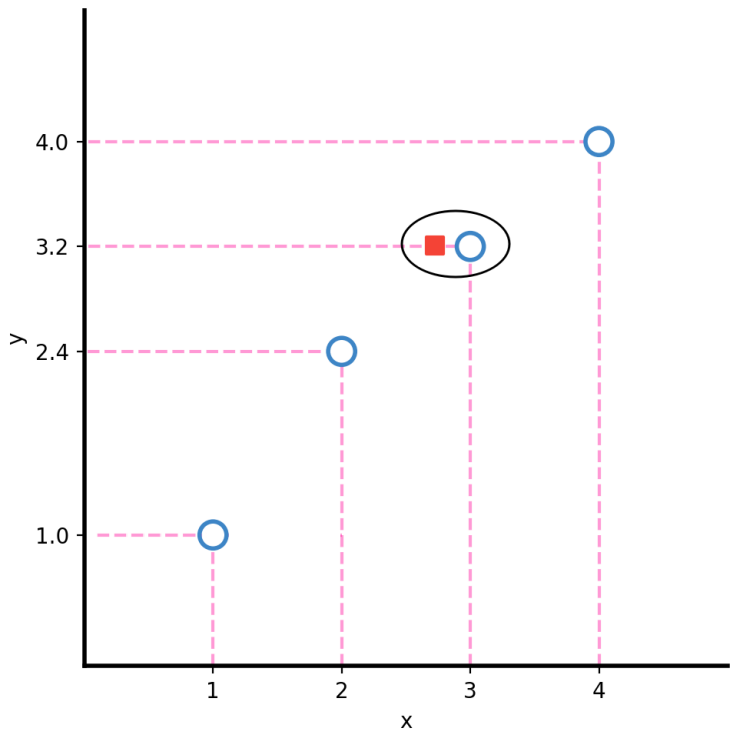
Se asigna entonces el valor de y de ese vecino.



Regresión con KNN: $K = 1$

Con $K = 1$, se toma solo un vecino.

Ese vecino tiene valor $y = 3{,}2$, por lo que el ■ adopta exactamente ese valor.



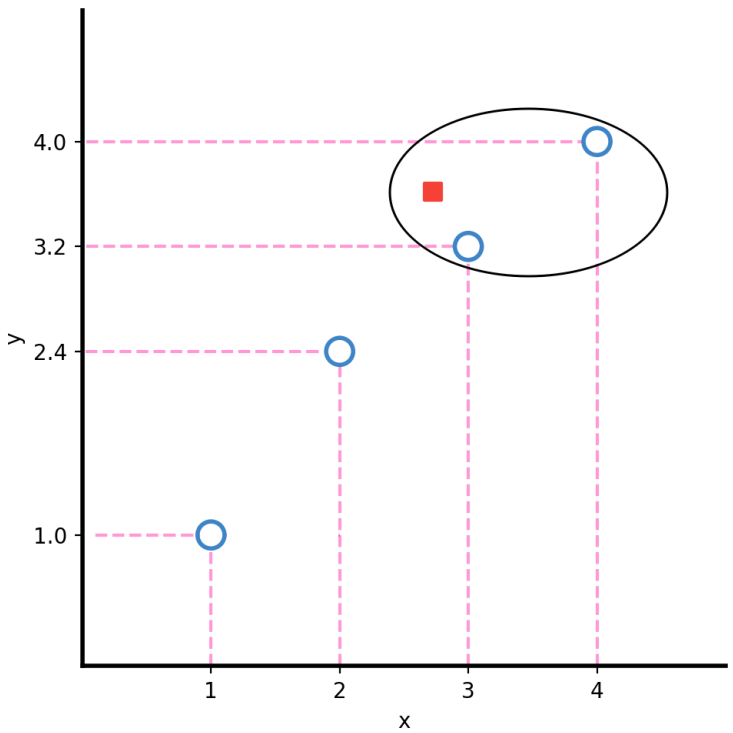
Regresión con KNN: $K = 2$

Con $K = 2$, se buscan los dos puntos más cercanos a el ■ .

Estos son los dos puntos a la derecha de el ■, con valores $y = 3{,}2$ y $y = 4$.

Se calcula el promedio:

$$y = \frac{3{,}2 + 4}{2} = 3{,}6$$



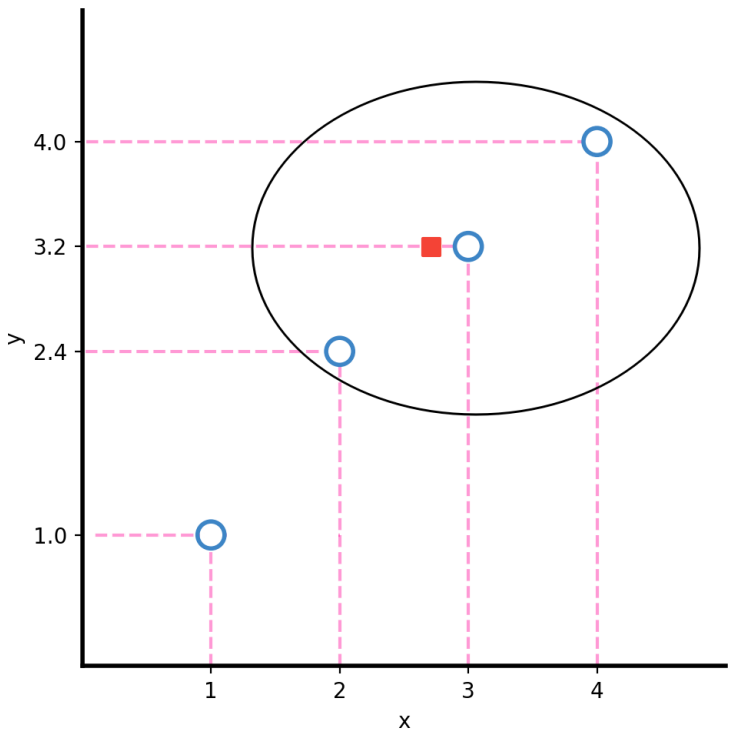
Regresión con KNN: $K = 3$

Con $K = 3$, se incluyen los dos vecinos a la derecha y uno a la izquierda de el ■ .

Sus valores de y son 3,2, 4 y 2,4.

La media es:

$$y = \frac{3{,}2 + 4 + 2{,}4}{3} = 3{,}2$$



Regresión con el dataset

Ahora el objetivo es predecir los minutos de supervivencia en el juego de *luz verde, luz roja*.
KNN promedia a los vecinos cercanos del jugador nuevo.

```
from sklearn.neighbors import KNeighborsRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

X = df[["Edad", "Deuda_millones"]]
y = df["Supervivencia_min"]

reg = make_pipeline(StandardScaler(),
                    KNeighborsRegressor(n_neighbors=3))
reg.fit(X, y)

nuevo = pd.DataFrame({"Edad":[38],
                      "Deuda_millones":[150]})
pred = reg.predict(nuevo)[0]
pred
```

Contenido

1. Aprendizaje Supervisado
2. Regresión
3. Clasificación
4. Prueba, ventajas y desventajas

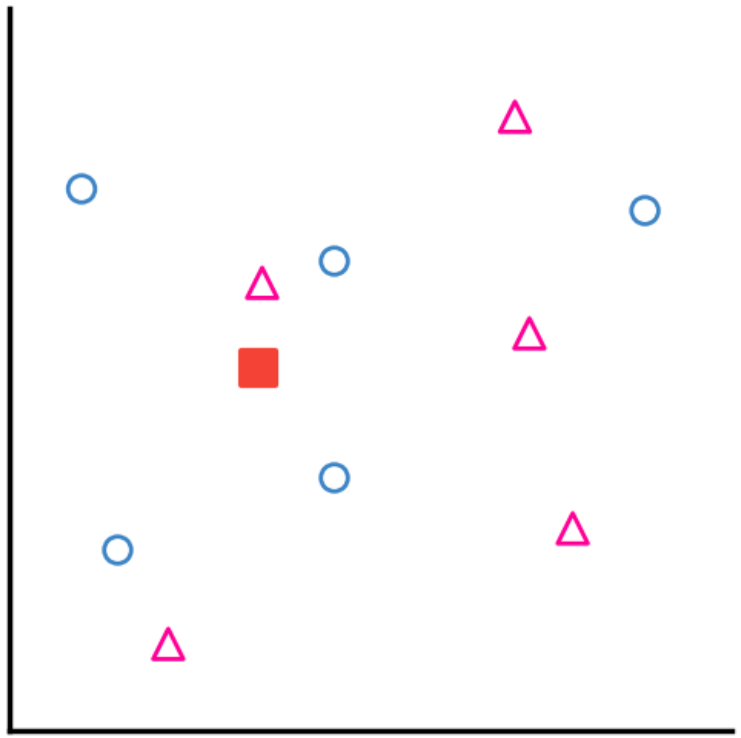
Clasificador KNN



¿Cómo funciona un clasificador KNN?

KNN es un algoritmo que clasifica una muestra según las clases de las muestras más cercanas a ella.

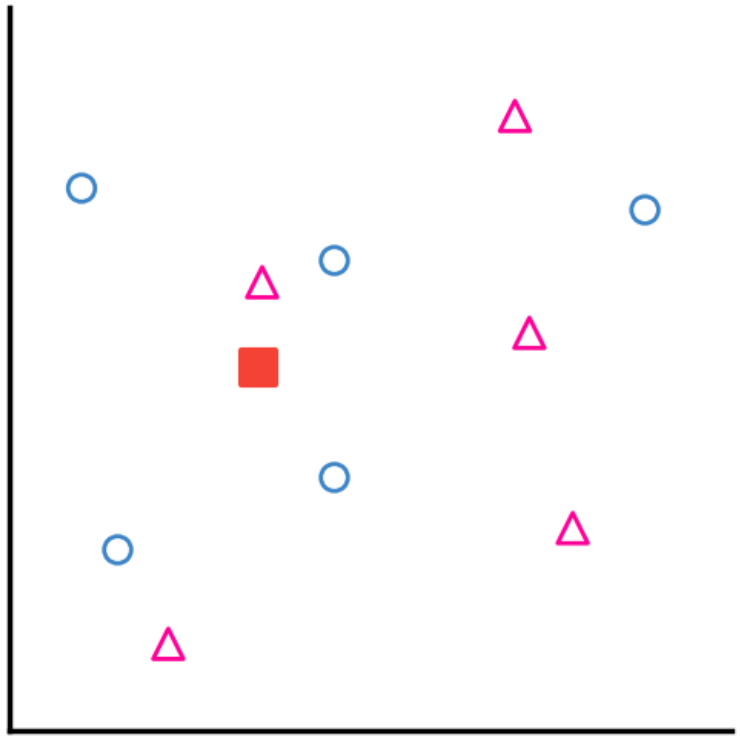
En el contexto de un problema de clasificación de 2 clases, representado en la figura, el objetivo es determinar la clase de la muestra que se representa con un ■ .



Clasificador KNN

En el enfoque empleado por KNN, el algoritmo comienza seleccionando primero un valor K, que representa el número de vecinos más cercanos.

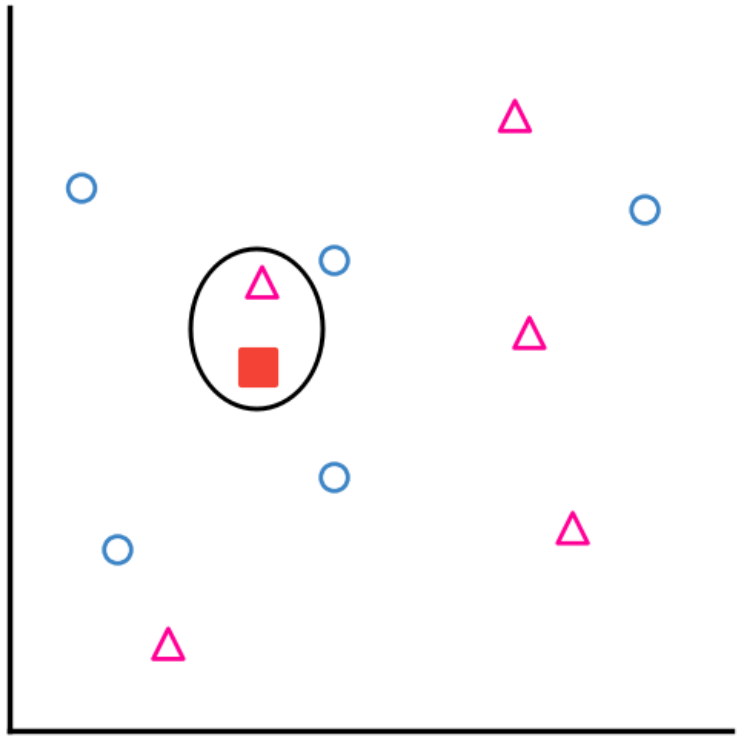
La determinación del valor de K recae en el programador.



Clasificador KNN con $K = 1$

Para este ejemplo, se consideran dos valores: 1 y 3.

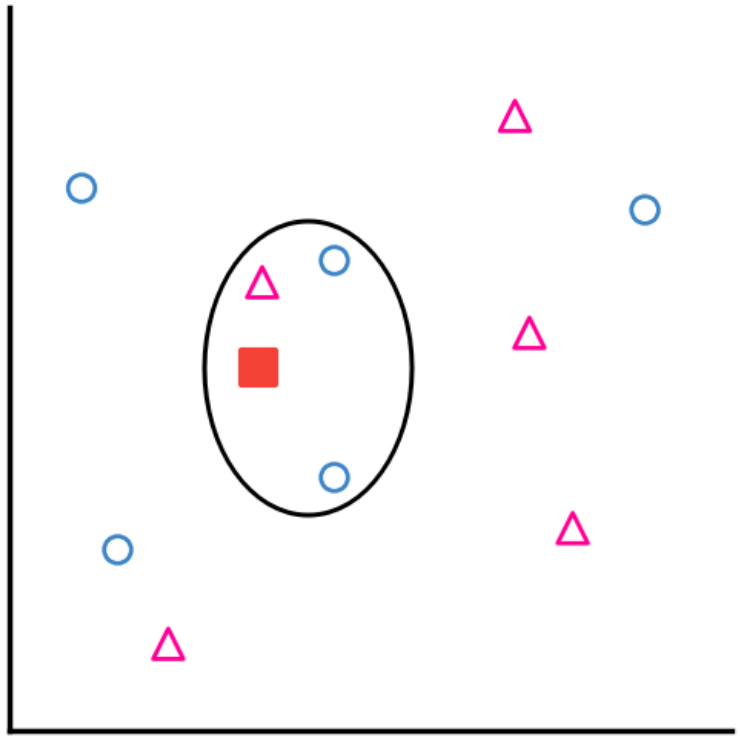
- Con $K = 1$, el algoritmo busca el vecino más cercano y evalúa su clase.
- En este caso particular, el vecino más cercano es un triángulo, por lo que la muestra se asigna a la categoría de triángulos.



Clasificador KNN con $K = 3$

Por otro lado, para $K = 3$:

- Uno de los vecinos es un triángulo, mientras que los otros dos son círculos.
- Dado que la clase de círculos predomina, la muestra se clasifica como un círculo.



Clasificación con el dataset

Etiquetamos grupos etarios (joven, adulto, mayor) y aplicamos KNN. El nuevo jugador de 38 años con 150 millones se clasifica según el voto de sus vecinos más cercanos.

```
from sklearn.neighbors import KNeighborsClassifier
import numpy as np

# clase etaria
bins = [0, 35, 55, 200]
labels = ["joven", "adulto", "mayor"]
df["GrupoEdad"] = pd.cut(df["Edad"], bins=bins,
                         labels=labels, right=False)

X = df[["Edad", "Deuda_millones"]].to_numpy()
y = df["GrupoEdad"].to_numpy()

clf = KNeighborsClassifier(n_neighbors=3)
clf.fit(X, y)

nuevo = np.array([[38, 150]])
pred = clf.predict(nuevo)[0]
pred
```

Contenido

1. Aprendizaje Supervisado
2. Regresión
3. Clasificación
4. Prueba, ventajas y desventajas

Checklist antes del juego final



Antes de soltar al algoritmo en datos reales:

- Definir bien la tarea: clasificación o regresión.
- Escalar características, codificar variables.
- Elegir la métrica de distancia adecuada.
- Seleccionar K con validación cruzada.
- Recordar límites: no extrapola, sufre en alta dimensión.

Parcial

Tu siguiente prueba consiste en
Hacer un Poster para el
Día de la Estadística



Ventajas de KNN

- **Fácil de implementar**: por su simplicidad y precisión inicial, es de los primeros clasificadores que aprende un científico de datos.
- **Se adapta fácilmente**: nuevos ejemplos de entrenamiento se incorporan sin necesidad de reentrenar, ya que todo se guarda en memoria.
- **Pocos hiperparámetros**: basta elegir K y una métrica de distancia, menos complejo que otros algoritmos.

Desventajas de KNN

- **No se escala bien**: ocupa mucha memoria y es costoso en tiempo y recursos, incluso con estructuras como Ball-Tree.
- **Maldición de la dimensionalidad**: el rendimiento cae cuando hay demasiadas variables irrelevantes o alta dimensión.
- **Propenso al sobreajuste**: K pequeños pueden sobreajustar, mientras que K grandes pueden subajustar.

Ball-Tree: estructura jerárquica que organiza los puntos en hiperesferas anidadas para acelerar la búsqueda de vecinos cercanos.

¿Clasificador GLM?

